BOOX STORE

Stripe Checkout: Code & PowerShell

A practical guide to the checkout code, the files it uses, and the exact Windows commands to run a safe test.

THE SIMPLE STORY

Browser sends book IDs + quantities ? server checks the catalogue ? Stripe takes test payment ? signed webhook confirms it ? store shows success

This PDF contains full listings of the checkout-specific source files. Secrets are never included. Keep real key values only in your ignored local server/.env file.

1. What happens during checkout

Step	What the code does	Where
1. Click checkout	The browser sends book IDs and quantities to the server. It does not send trusted prices.	CartSection.jsx
2. Check shopper	The server asks Better Auth whether this request has a valid signed-in user.	checkout-routes.js
3. Check basket and prices	It validates the basket and reads current catalogue prices from PostgreSQL, then computes integer cents.	cart-validation.js + checkout-routes.js
4. Save order	One database transaction saves a pending order and its line items.	Drizzle schema + checkout-routes.js
5. Open Stripe Checkout	Stripe creates a hosted payment page; the browser is redirected to its HTTPS URL.	checkout-routes.js + CartSection.jsx
6. Confirm payment	A raw webhook body and Stripe signature are checked; a matching paid session updates the order.	/api/stripe/webhook
7. Show result	The browser asks the server for this signed-in user?s order. The cart clears only after status is paid.	CartSection.jsx + GET route

Why the browser never decides the price

Browser values can be changed by the person using the browser. The browser sends only book IDs and quantities. The server fetches the real books and prices itself, computes the total in cents, and sends those prices to Stripe.

Why a webhook decides ?paid?

A return URL or green check on a payment page is not proof. Only a correctly signed event from Stripe can change a pending order to paid. The webhook also checks the order ID, Checkout Session ID, currency, and total.

Demo rules

This project requires a signed-in customer. It uses USD and charges no shipping or tax. Use Stripe test mode only for the walkthrough; test card data does not move real money.

2. The file map

File	Purpose
frontend/src/components/CartSection.jsx	Cart UI, API request, redirect, return-status checks, success screen, and clear-after-paid behavior.
frontend/src/context/CartContext.jsx	Shared browser cart state and clearCart() used after confirmed payment.
frontend/src/context/cart-context.js	React Context and useCart() helper used by cart components.

server/src/checkout-routes.js	Create Checkout Session, look up the owner?s order, verify/process Stripe webhooks.
server/src/cart-validation.js	Validate untrusted cart JSON.
server/src/cart-validation.test.js	Automated tests for valid and invalid carts.
server/src/db/schema.js	Drizzle definitions for books, orders, and order_items.
server/src/db/migrations/0001_create_orders.sql	SQL to create the order tables when setting up a new database.
server/src/index.js	Mounts checkout routes into Express.
frontend/vite.config.js	Forwards local /api calls from Vite to the Express server.

3. PowerShell setup: safe and step by step

Commands below assume the repository is at C:\Users\louis\Documents\bookstore-practicum. If yours is elsewhere, change that folder path. Run them in PowerShell; the .cmd suffix avoids PowerShell script-policy confusion.

Step A - install Stripe CLI (once)

```
npm.cmd install --global --allow-scripts=@stripe/cli @stripe/cli
npx.cmd --yes @stripe/cli version
```

The first command installs Stripe CLI and explicitly permits its install script. The second runs the CLI through npm without relying on the short ?stripe? command being on PATH.

Step B - connect the CLI to your Stripe test account (once)

```
npx.cmd --yes @stripe/cli login
```

Follow the browser approval prompt. Do not share the pairing code. This authorizes the CLI to access the Stripe account you selected.

Step C - place the two secret values in server/.env

```
Set-Location 'C:\Users\louis\Documents\bookstore-practicum\server'
notepad .env
```

Keep existing database/auth settings. Locally, make sure these two lines exist and each has the matching value: STRIPE_SECRET_KEY=your Stripe test secret key (sk_test_...) and STRIPE_WEBHOOK_SECRET=your CLI signing secret (whsec_...). Replace the description with the real value in Notepad; descriptions are not valid keys. Save the file. Never paste the values into chat or commit the file.

Step D - run the webhook listener

```
npx.cmd --yes @stripe/cli listen --forward-to localhost:3000/api/stripe/webhook --events checkout.session.completed,checkout.session.async_payment_succeeded,checkout.session.async_payment_failed,checkout.s
```

The command keeps running and prints a webhook signing secret beginning with whsec_. Put that exact value in STRIPE_WEBHOOK_SECRET in server/.env. Keep this listener terminal open. If you restart the listener and it gives a different secret, update .env and restart the API.

Step E - start the API and frontend in separate PowerShell windows

```
# PowerShell window 2: server
Set-Location 'C:\Users\louis\Documents\bookstore-practicum\server'
npm.cmd run dev

# PowerShell window 3: frontend
Set-Location 'C:\Users\louis\Documents\bookstore-practicum\frontend'
npm.cmd run dev
```

Keep all three windows open: (1) Stripe listener, (2) server, and (3) frontend. Open the Vite URL, usually http://localhost:5173, and sign in.

The repository?s configured database already contains the order tables. Do not run the migration on that database. The SQL listing later in this PDF is for a new database that does not yet have those tables.

4. Run a test payment and check the result

```
1. Sign in to the local bookstore.
2. Add one or more books to the cart.
3. Click ?Proceed to secure checkout?.
4. On Stripe test checkout, use card: 4242 4242 4242 4242
   Expiry: any future month/year; CVC: any 3 digits.
5. Submit and return to the store.
6. In the Stripe listener window, look for checkout.session.completed
   and a forwarded HTTP [200] response.
7. The store should show Payment confirmed and an order number.
   The cart clears only after the backend reports paid.
```

If the store never confirms payment

Symptom	Check
Checkout says Stripe not configured	Confirm the server has STRIPE_SECRET_KEY in server/.env and restart it.
Webhook signature rejected (HTTP 400)	The server value must exactly match the whsec_ value printed by the currently running CLI listener. Restart the server after changing .env.
Webhook response HTTP 503	One or both Stripe environment settings are missing in the server process.
Webhook response HTTP 500	Read the server terminal for the processing error; don?t share secrets.
Payment page returns but order pending	Check that the listener received checkout.session.completed and forwarded it to the correct port/path.
Sign-in required	Sign in before starting checkout and return using the same account.

5. What can safely be committed?

Source code, migration SQL, tests, and this guide can be committed. The real server/.env must stay local and ignored by Git. The repository has an .env.example with blank Stripe settings for showing the variable names only.

Never put sk_test_..., sk_live_..., or whsec_... values in a source file, README, PDF, screenshot, or Git commit. If a real secret is accidentally shared, revoke/rotate it in Stripe.

6. Complete checkout-related code listings

The following pages reproduce the current committed files verbatim (with printed line numbers and visual wrapping). Use them to read or compare the implementation. The environment template contains only blank placeholders; no local .env file is included.

Server checkout and webhook routes

The main server file. Read it with the explanations above: session check, trusted prices, pending order, Stripe session, owner-only status lookup, and signed webhook.

Complete source: server/src/checkout-routes.js | 374 lines

```
import express from 'express'
import Stripe from 'stripe'
import { fromNodeHeaders } from 'better-auth/node'
import { and, eq, inArray } from 'drizzle-orm'
import { auth } from './auth.js'
import { db } from './db/index.js'
import { books, orderItems, orders } from './db/schema.js'
import { parseCartItems } from './cart-validation.js'

const router = express.Router()
const currency = 'usd'
let stripeClient

function getStripeClient() {
  if (!process.env.STRIPE_SECRET_KEY) {
    const error = new Error('Stripe is not configured on the server.')
    error.statusCode = 503
    throw error
  }

  if (!stripeClient) {
    stripeClient = new Stripe(process.env.STRIPE_SECRET_KEY)
  }

  return stripeClient
}

function getStoreUrl() {
  const configuredUrl =
    process.env.RENDER_EXTERNAL_URL || process.env.CLIENT_URL

  if (!configuredUrl) {
    const error = new Error('The store URL is not configured on the server.')
    error.statusCode = 503
    throw error
  }

  const url = new URL(configuredUrl)
  const isLocalhost = ['localhost', '127.0.0.1'].includes(url.hostname)

  if (url.protocol !== 'https:' && !(url.protocol === 'http:' && isLocalhost)) {
    const error = new Error('The configured store URL must use HTTPS.')
    error.statusCode = 503
    throw error
  }

  return url.origin
}

async function getSignedInUser(req) {
  const session = await auth.api.getSession({
    headers: fromNodeHeaders(req.headers),
  })

  return session?.user ?? null
}

router.post(
  '/checkout/session',
  express.json({ limit: '16kb' }),
```

```
async (req, res) => {
  try {
    const user = await getSignedInUser(req)

    if (!user) {
      return res.status(401).json({
        error: 'Please sign in before checking out.',
      })
    }

    const cartItems = parseCartItems(req.body)
    const stripe = getStripeClient()
    const storeUrl = getStoreUrl()
    const requestedBookIds = cartItems.map((item) => item.bookId)
    const catalogueBooks = await db
      .select()
      .from(books)
      .where(inArray(books.id, requestedBookIds))

    if (catalogueBooks.length !== requestedBookIds.length) {
      return res.status(400).json({
        error: 'A book in your cart is no longer available.',
      })
    }

    const booksById = new Map(
      catalogueBooks.map((book) => [book.id, book]),
    )
    const orderLines = cartItems.map((item) => {
      const book = booksById.get(item.bookId)
      const unitPriceCents = Math.round(book.price * 100)

      if (!Number.isSafeInteger(unitPriceCents) || unitPriceCents < 1) {
        throw new Error(`Book ${item.bookId} has an invalid price.`)
      }

      return {
        book,
        quantity: item.quantity,
        unitPriceCents,
      }
    })
    const totalCents = orderLines.reduce(
      (total, item) => total + item.unitPriceCents * item.quantity,
      0,
    )

    if (!Number.isSafeInteger(totalCents) || totalCents > 2147483647) {
      return res.status(400).json({
        error: 'Your cart total is too large to check out.',
      })
    }

    const [order] = await db.transaction(async (tx) => {
      const [createdOrder] = await tx
        .insert(orders)
        .values({
          userId: user.id,
          totalCents,
        })
        .returning({ id: orders.id })

      await tx.insert(orderItems).values(
        orderLines.map((item) => ({
          orderId: createdOrder.id,
          bookId: item.book.id,
          quantity: item.quantity,
          unitPriceCents: item.unitPriceCents,
```

```
          })),
        )

        return [createdOrder]
      })

      let checkoutSession

      try {
        checkoutSession = await stripe.checkout.sessions.create({
          mode: 'payment',
          customer_email: user.email,
          client_reference_id: String(order.id),
          metadata: { orderId: String(order.id) },
          line_items: orderLines.map((item) => ({
            quantity: item.quantity,
            price_data: {
              currency,
              unit_amount: item.unitPriceCents,
              product_data: {
                name: item.book.title,
                description: `By ${item.book.author}`,
              },
            },
          })),
          success_url: `${storeUrl}/?checkout=success&session_id={CHECKOUT_SESSION_ID}#cart`,
          cancel_url: `${storeUrl}/?checkout=cancel#cart`,
        })

        if (!checkoutSession.url) {
          throw new Error('Stripe did not return a Checkout URL.')
        }

        const updatedOrders = await db
          .update(orders)
          .set({ stripeSessionId: checkoutSession.id })
          .where(
            and(eq(orders.id, order.id), eq(orders.status, 'pending')),
          )
          .returning({ id: orders.id })

        if (updatedOrders.length !== 1) {
          throw new Error('Could not link the Stripe session to the order.')
        }

        return res.status(201).json({ url: checkoutSession.url })
      } catch (error) {
        if (checkoutSession?.id) {
          try {
            await stripe.checkout.sessions.expire(checkoutSession.id)
          } catch (expireError) {
            console.error(
              'Could not expire an unlinked Stripe Checkout Session:',
              expireError.message,
            )
          }
        }

        try {
          await db.delete(orders).where(eq(orders.id, order.id))
        } catch (cleanupError) {
          console.error(
            `Could not remove incomplete order ${order.id}:`,
            cleanupError.message,
          )
        }

        throw error
```

```
      }
    } catch (error) {
      console.error('Could not create checkout session:', error.message)
      return res.status(error.statusCode || 500).json({
        error:
          error.statusCode && (error.statusCode < 500 || error.statusCode === 503)
            ? error.message
            : 'Could not start checkout. Please try again.',
      })
    }
  },
)

router.get('/checkout/session/:sessionId', async (req, res) => {
  try {
    const user = await getSignedInUser(req)

    if (!user) {
      return res.status(401).json({
        error: 'Please sign in to view this order.',
      })
    }

    const sessionId = req.params.sessionId

    if (!/^cs_(test|live)_[A-Za-z0-9]+$/.test(sessionId)) {
      return res.status(400).json({ error: 'Invalid Checkout Session ID.' })
    }

    const [order] = await db
      .select({
        id: orders.id,
        status: orders.status,
        totalCents: orders.totalCents,
      })
      .from(orders)
      .where(
        and(
          eq(orders.stripeSessionId, sessionId),
          eq(orders.userId, user.id),
        ),
      )
      .limit(1)

    if (!order) {
      return res.status(404).json({ error: 'Order not found.' })
    }

    return res.json({
      orderId: order.id,
      status: order.status,
      totalCents: order.totalCents,
    })
  } catch (error) {
    console.error('Could not look up checkout order:', error.message)
    return res.status(500).json({
      error: 'Could not check your order. Please try again.',
    })
  }
})

router.post(
  '/stripe/webhook',
  express.raw({ type: 'application/json', limit: '1mb' }),
  async (req, res) => {
    if (!process.env.STRIPE_SECRET_KEY || !process.env.STRIPE_WEBHOOK_SECRET) {
      return res.status(503).json({
        error: 'Stripe payments are not configured on the server.',
```

```
      })
    }

    const signature = req.headers['stripe-signature']

    if (typeof signature !== 'string' || !Buffer.isBuffer(req.body)) {
      return res.status(400).json({ error: 'Invalid Stripe webhook request.' })
    }

    let event

    try {
      event = getStripeClient().webhooks.constructEvent(
        req.body,
        signature,
        process.env.STRIPE_WEBHOOK_SECRET,
      )
    } catch (error) {
      console.error('Could not verify Stripe webhook:', error.message)
      return res.status(400).json({ error: 'Invalid Stripe webhook signature.' })
    }

    const paidEventTypes = new Set([
      'checkout.session.completed',
      'checkout.session.async_payment_succeeded',
    ])
    const expiredEvent = event.type === 'checkout.session.expired'
    const failedEvent = event.type === 'checkout.session.async_payment_failed'

    if (
      !paidEventTypes.has(event.type) &&
      !expiredEvent &&
      !failedEvent
    ) {
      return res.json({ received: true })
    }

    try {
      const checkoutSession = event.data.object
      const orderId = Number(checkoutSession.metadata?.orderId)

      if (!Number.isSafeInteger(orderId) || orderId < 1) {
        console.error('Stripe Checkout Session has invalid order metadata.')
        return res.status(400).json({ error: 'Invalid order metadata.' })
      }

      const [order] = await db
        .select()
        .from(orders)
        .where(eq(orders.id, orderId))
        .limit(1)

      if (!order || order.stripeSessionId !== checkoutSession.id) {
        console.error(
          `Stripe Checkout Session ${checkoutSession.id} does not match order ${orderId}.`,
        )
        return res.status(400).json({ error: 'Checkout order did not match.' })
      }

      if (paidEventTypes.has(event.type)) {
        if (checkoutSession.payment_status !== 'paid') {
          return res.json({ received: true })
        }

        if (
          checkoutSession.amount_total !== order.totalCents ||
          checkoutSession.currency !== currency
        ) {
```

```
          console.error(
            `Stripe amount or currency did not match order ${order.id}.`,
          )
          return res.status(400).json({ error: 'Checkout total did not match.' })
        }

        if (order.status === 'pending') {
          await db
            .update(orders)
            .set({ status: 'paid', paidAt: new Date() })
            .where(
              and(
                eq(orders.id, order.id),
                eq(orders.status, 'pending'),
                eq(orders.stripeSessionId, checkoutSession.id),
              ),
            )
        }
      } else if (order.status === 'pending') {
        await db
          .update(orders)
          .set({
            status: expiredEvent ? 'expired' : 'failed',
          })
          .where(
            and(
              eq(orders.id, order.id),
              eq(orders.status, 'pending'),
              eq(orders.stripeSessionId, checkoutSession.id),
            ),
          )
      }

      return res.json({ received: true })
    } catch (error) {
      console.error('Could not process Stripe webhook:', error.message)
      return res.status(500).json({ error: 'Could not process webhook.' })
    }
  },
)

export default router
```

Cart input validation

The server treats browser JSON as untrusted. This helper rejects empty baskets, duplicate IDs, invalid IDs, and quantities that are not positive whole numbers.

Complete source: server/src/cart-validation.js | 34 lines

```
export function parseCartItems(body) {
  if (!body || !Array.isArray(body.items) || body.items.length === 0) {
    const error = new Error('Your cart is empty.')
    error.statusCode = 400
    throw error
  }

  if (body.items.length > 100) {
    const error = new Error('Your cart has too many different books.')
    error.statusCode = 400
    throw error
  }

  const seenBookIds = new Set()
  return body.items.map((item) => {
    if (
      !item ||
      typeof item !== 'object' ||
      !Number.isInteger(item.bookId) ||
      item.bookId < 1 ||
      !Number.isInteger(item.quantity) ||
      item.quantity < 1 ||
      item.quantity > 999999 ||
      seenBookIds.has(item.bookId)
    ) {
      const error = new Error('Your cart contains an invalid book or quantity.')
      error.statusCode = 400
      throw error
    }

    seenBookIds.add(item.bookId)
    return { bookId: item.bookId, quantity: item.quantity }
  })
}
```

Cart validation tests

These tests check accepted cart data and malformed cases. Run them with npm.cmd test from the server folder.

Complete source: server/src/cart-validation.test.js | 52 lines

```
import assert from 'node:assert/strict'
import test from 'node:test'
import { parseCartItems } from './cart-validation.js'

test('accepts a list of unique books with positive whole-number quantities', () => {
  assert.deepEqual(
    parseCartItems({
      items: [
        { bookId: 2, quantity: 3 },
        { bookId: 8, quantity: 1 },
      ],
    }),
    [
      { bookId: 2, quantity: 3 },
      { bookId: 8, quantity: 1 },
    ],
  )
})

test('rejects an empty cart', () => {
  assert.throws(
    () => parseCartItems({ items: [] }),
    { message: 'Your cart is empty.', statusCode: 400 },
  )
})

test('rejects invalid quantities and duplicate books', () => {
  for (const items of [
    [{ bookId: 1, quantity: 0 }],
    [{ bookId: 1, quantity: 1.5 }],
    [{ bookId: 1, quantity: 1000000 }],
    [
      { bookId: 1, quantity: 1 },
      { bookId: 1, quantity: 2 },
    ],
    [null],
  ]) {
    assert.throws(() => parseCartItems({ items }), { statusCode: 400 })
  }
})

test('rejects carts with too many distinct books', () => {
  const items = Array.from({ length: 101 }, (_, index) => ({
    bookId: index + 1,
    quantity: 1,
  }))

  assert.throws(
    () => parseCartItems({ items }),
    { message: 'Your cart has too many different books.', statusCode: 400 },
  )
})
```

Cart, checkout button, return state, and success screen

This complete component connects the visible cart to the server. It sends IDs/quantities, validates the Stripe redirect host, retries pending status briefly, clears the cart after a paid response, and presents the persistent confirmation screen.

Complete source: frontend/src/components/CartSection.jsx | 418 lines

```
import { useEffect, useState } from 'react'
import QuantityStepper from './QuantityStepper.jsx'
import { useCart } from '../context/cart-context.js'

const money = new Intl.NumberFormat('en-US', {
  style: 'currency',
  currency: 'USD',
})

export default function CartSection({ books, booksLoading, booksError }) {
  const {
    items,
    itemCount,
    storageError,
    setItemQuantity,
    removeItem,
    clearCart,
  } = useCart()
  const [checkoutBusy, setCheckoutBusy] = useState(false)
  const [checkoutError, setCheckoutError] = useState('')
  const [checkoutRetry, setCheckoutRetry] = useState(0)
  const [checkoutFeedback, setCheckoutFeedback] = useState(() => {
    const params = new URLSearchParams(window.location.search)
    const result = params.get('checkout')
    const sessionId = params.get('session_id')

    if (result === 'cancel') {
      return {
        status: 'cancelled',
        message: 'Checkout was cancelled. Your cart is still here.',
      }
    }

    if (result === 'success' && sessionId) {
      return {
        status: 'checking',
        message: 'Checking your payment with the store...',
        sessionId,
      }
    }

    return null
  })

  const cartItems = items.map((item) => ({
    ...item,
    book: books.find((book) => book.id === item.bookId),
  }))
  const subtotalCents = cartItems.reduce(
    (total, item) =>
      total +
      (item.book ? Math.round(item.book.price * 100) * item.quantity : 0),
    0,
  )
  const hasUnavailableBooks =
    !booksLoading &&
    !booksError &&
    cartItems.some((item) => !item.book)
```

```
  const canCheckout =
    items.length > 0 &&
    !booksLoading &&
    !booksError &&
    !hasUnavailableBooks &&
    !checkoutBusy

  useEffect(() => {
    if (checkoutFeedback?.status !== 'checking') {
      return undefined
    }

    const controller = new AbortController()
    let retryTimer

    async function checkPayment() {
      try {
        const response = await fetch(
          `/api/checkout/session/${encodeURIComponent(checkoutFeedback.sessionId)}`,
          { signal: controller.signal },
        )
        const result = await response.json()

        if (!response.ok) {
          setCheckoutFeedback({
            status: 'error',
            message:
              response.status === 401
                ? 'Sign in to the same account you used to place the order, then check again.'
                : result.error || 'Could not check the payment status.',
            sessionId: checkoutFeedback.sessionId,
          })
          return
        }

        if (result.status === 'paid') {
          clearCart()
          setCheckoutFeedback({
            status: 'paid',
            message: `Payment confirmed for order ${result.orderId}. Thank you!`,
            orderId: result.orderId,
            totalCents: result.totalCents,
          })
          return
        }

        if (['failed', 'expired'].includes(result.status)) {
          setCheckoutFeedback({
            status: result.status,
            message:
              result.status === 'expired'
                ? 'This checkout session expired. Your cart is still here; you can try again.'
                : 'The payment did not complete. Your cart is still here; you can try again.',
            sessionId: checkoutFeedback.sessionId,
          })
          return
        }

        if (checkoutRetry < 5) {
          retryTimer = window.setTimeout(
            () => setCheckoutRetry((attempt) => attempt + 1),
            2000,
          )
        } else {
          setCheckoutFeedback({
            status: 'pending',
            message:
              'Payment is still being confirmed. You can check again in a moment.',
```

```
            sessionId: checkoutFeedback.sessionId,
          })
        }
      } catch (error) {
        if (!controller.signal.aborted) {
          console.error('Could not check payment status:', error.message)
          setCheckoutFeedback({
            status: 'error',
            message: 'Could not check the payment status. Please try again.',
            sessionId: checkoutFeedback.sessionId,
          })
        }
      }
    }

    checkPayment()
    return () => {
      controller.abort()
      window.clearTimeout(retryTimer)
    }
  }, [checkoutFeedback, checkoutRetry, clearCart])

  async function startCheckout() {
    setCheckoutBusy(true)
    setCheckoutError('')

    try {
      const response = await fetch('/api/checkout/session', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          items: items.map(({ bookId, quantity }) => ({ bookId, quantity })),
        }),
      })
      const result = await response.json()

      if (!response.ok) {
        throw new Error(
          response.status === 401
            ? 'Please sign in before checking out.'
            : result.error || 'Could not start checkout. Please try again.',
        )
      }

      const checkoutUrl = new URL(result.url)

      if (
        checkoutUrl.protocol !== 'https:' ||
        checkoutUrl.hostname !== 'checkout.stripe.com'
      ) {
        throw new Error('Stripe returned an unexpected checkout address.')
      }

      window.location.assign(checkoutUrl.href)
    } catch (error) {
      console.error('Could not start checkout:', error.message)
      setCheckoutError(error.message)
      setCheckoutBusy(false)
    }
  }

  function continueBrowsing() {
    const url = new URL(window.location.href)
    url.searchParams.delete('checkout')
    url.searchParams.delete('session_id')
    url.hash = 'shop'
    window.history.replaceState(window.history.state, '', url)
    setCheckoutFeedback(null)
```

```
    document.getElementById('shop')?.scrollIntoView({ behavior: 'smooth' })
  }

  return (
    <>
      {checkoutFeedback?.status === 'paid' && (
        <div className="fixed inset-0 z-50 flex min-h-screen items-center justify-center overflow-y-auto bg-[#F2E6D2] px-5 py-10 text-[#35251C]">
          <section
            role="status"
            aria-labelledby="checkout-success-title"
            className="w-full max-w-xl rounded-2xl border border-[#CDB99A] bg-[#FFF8EC] p-8 text-center shadow-xl sm:p-12"
          >
            <div
              aria-hidden="true"
              className="mx-auto flex h-20 w-20 items-center justify-center rounded-full bg-green-100 text-5xl font-bold text-green-800"
            >
              ✓
            </div>
            <p className="mt-6 text-sm font-bold uppercase tracking-widest text-[#6B5746]">
              Boox Store
            </p>
            <h1
              id="checkout-success-title"
              className="mt-3 text-3xl font-bold sm:text-4xl"
            >
              Payment confirmed!
            </h1>
            <p className="mt-4 leading-relaxed text-[#6B5746]">
              Thank you for supporting our bookstore. Your order is safely
              recorded and your cart is clear.
            </p>
            <div className="mt-6 rounded-lg bg-[#F2E6D2] p-4">
              <p className="font-bold">Order #{checkoutFeedback.orderId}</p>
              <p className="mt-1 text-[#6B5746]">
                Paid {money.format(checkoutFeedback.totalCents / 100)}
              </p>
            </div>
            <button
              type="button"
              onClick={continueBrowsing}
              className="mt-8 rounded-lg bg-[#35251C] px-6 py-3 font-bold text-[#FFF8EC] hover:bg-[#513A2A] focus-visible:outline-2 focus-visible:outline-offset-4 focus-visible:outline-[#35251C]"
            >
              Continue browsing
            </button>
          </section>
        </div>
      )}
      <section
        id="cart"
        className="mx-auto max-w-6xl scroll-mt-24 px-5 py-12 text-[#35251C]"
      >
      <div className="rounded-xl border border-[#CDB99A] bg-[#E6D5B8] p-6 shadow-md">
        <div className="flex flex-wrap items-center justify-between gap-4">
          <div>
            <p className="mb-2 text-sm font-bold uppercase tracking-widest text-[#6B5746]">
              Your shopping basket
            </p>
            <h2 className="text-3xl font-bold">Your cart</h2>
            <p className="mt-2 text-[#6B5746]" role="status">
              {itemCount === 1 ? '1 book in your cart' : `${itemCount} books in your cart`}
            </p>
          </div>

          {items.length > 0 && (
```

```
            <button
              type="button"
              onClick={clearCart}
              className="rounded-lg border border-[#80664F] px-4 py-2 font-bold hover:bg-[#F2E6D2]"
            >
              Empty cart
            </button>
          )}
        </div>

        {storageError && (
          <p role="alert" className="mt-4 text-red-800">
            {storageError}
          </p>
        )}

        {hasUnavailableBooks && (
          <p role="alert" className="mt-4 text-red-800">
            Some books in your saved cart are no longer in the catalogue. You can remove them below.
          </p>
        )}
        {booksError && items.length > 0 && (
          <p role="alert" className="mt-4 text-red-800">
            Book details could not be loaded. Your saved cart is still here; refresh to try again.
          </p>
        )}
        {checkoutFeedback && (
          <div
            role={checkoutFeedback.status === 'error' ? 'alert' : 'status'}
            className={`mt-4 rounded-lg p-4 ${
              checkoutFeedback.status === 'error'
                ? 'bg-red-100 text-red-900'
                : 'bg-[#FFF8EC]'
            }`}
          >
            <p>{checkoutFeedback.message}</p>
            {['pending', 'error', 'expired', 'failed'].includes(
              checkoutFeedback.status,
            ) && checkoutFeedback.sessionId && (
              <button
                type="button"
                onClick={() => {
                  setCheckoutFeedback((current) => ({
                    ...current,
                    status: 'checking',
                  }))
                  setCheckoutRetry(0)
                }}
                className="mt-2 font-bold underline underline-offset-4"
              >
                Check payment again
              </button>
            )}
          </div>
        )}

        {items.length === 0 ? (
          <p className="mt-6 rounded-lg bg-[#FFF8EC] p-4">
            Your cart is empty. Add a book from the collection to see it here.
          </p>
        ) : (
          <>
            <ul className="mt-6 divide-y divide-[#CDB99A]">
              {cartItems.map((item) => (
                <li
                  key={item.bookId}
                  className="flex flex-wrap items-center gap-4 py-4 first:pt-0 last:pb-0"
                >
```

```
                  {item.book ? (
                    <>
                      <img
                        src={item.book.image}
                        alt=""
                        className="h-20 w-16 rounded bg-[#FFF8EC] object-contain p-1"
                      />
                      <div className="min-w-40 flex-1">
                        <h3 className="font-bold">{item.book.title}</h3>
                        <p className="text-sm text-[#6B5746]">
                          {money.format(item.book.price)} each
                        </p>
                      </div>
                      <QuantityStepper
                        quantity={item.quantity}
                        onQuantityChange={(quantity) =>
                          setItemQuantity(item.bookId, quantity)
                        }
                        label={item.book.title}
                      />
                      <p className="w-24 text-right font-bold">
                        {money.format(
                          (Math.round(item.book.price * 100) * item.quantity) /
                            100,
                        )}
                      </p>
                    </>
                  ) : (
                    <>
                      <p className="min-w-40 flex-1 font-bold">
                        {booksLoading
                          ? 'Loading book details...'
                          : booksError
                            ? 'Book details unavailable'
                            : `Unavailable book (ID ${item.bookId})`}
                      </p>
                      <span>Quantity: {item.quantity}</span>
                    </>
                  )}
                  <button
                    type="button"
                    onClick={() => removeItem(item.bookId)}
                    className="rounded px-3 py-2 font-bold underline underline-offset-4 hover:bg-[#F2E6D2]"
                    aria-label={`Remove ${item.book?.title ?? 'unavailable book'} from cart`}
                  >
                    Remove
                  </button>
                </li>
              ))}
            </ul>

            <div className="mt-6 border-t border-[#CDB99A] pt-4 text-right">
              <p className="text-lg font-bold">
                Subtotal: {money.format(subtotalCents / 100)}
              </p>
              <p className="mt-1 text-sm text-[#6B5746]">
                Demo checkout uses USD and does not add shipping or tax.
              </p>
              <button
                type="button"
                onClick={startCheckout}
                disabled={!canCheckout}
                className="mt-4 rounded-lg bg-[#35251C] px-5 py-3 font-bold text-[#FFF8EC] hover:bg-[#513A2A]
disabled:cursor-not-allowed disabled:opacity-50"
              >
                {checkoutBusy
                  ? 'Opening secure checkout...'
                  : 'Proceed to secure checkout'}
```

```
              </button>
              {checkoutError && (
                <p role="alert" className="mt-3 text-red-800">
                  {checkoutError}
                  {checkoutError.includes('sign in') && (
                    <>
                      {' '}
                      <a
                        href="#account"
                        className="font-bold underline underline-offset-4"
                      >
                        Go to your account
                      </a>
                    </>
                  )}
                </p>
              )}
            </div>
          </>
        )}
      </div>
      </section>
    </>
  )
}
```

Shared cart state used by checkout

This provider stores the browser cart and exposes clearCart(), called only when the server reports a paid order.

Complete source: frontend/src/context/CartContext.jsx | 116 lines

```
import { useCallback, useState } from 'react'
import { CartContext } from './cart-context.js'

const CART_STORAGE_KEY = 'boox-store-cart'

function readStoredCart() {
  try {
    const storedCart = window.localStorage.getItem(CART_STORAGE_KEY)

    if (!storedCart) {
      return { items: [], storageError: '' }
    }

    const parsedCart = JSON.parse(storedCart)

    if (!Array.isArray(parsedCart)) {
      throw new Error('Stored cart is not a list.')
    }

    const seenBookIds = new Set()
    const items = parsedCart.map((item) => {
      if (
        !Number.isInteger(item.bookId) ||
        item.bookId < 1 ||
        !Number.isInteger(item.quantity) ||
        item.quantity < 1 ||
        seenBookIds.has(item.bookId)
      ) {
        throw new Error('Stored cart contains an invalid item.')
      }

      seenBookIds.add(item.bookId)
      return { bookId: item.bookId, quantity: item.quantity }
    })

    return { items, storageError: '' }
  } catch (error) {
    console.error('Could not restore the saved cart:', error)
    return {
      items: [],
      storageError: 'Your saved cart could not be read in this browser.',
    }
  }
}

export function CartProvider({ children }) {
  const [cart, setCart] = useState(readStoredCart)
  const { items, storageError } = cart

  function saveItems(nextItems) {
    try {
      window.localStorage.setItem(CART_STORAGE_KEY, JSON.stringify(nextItems))
      setCart({ items: nextItems, storageError: '' })
    } catch (error) {
      console.error('Could not save the cart:', error)
      setCart({
        items: nextItems,
        storageError: 'Your cart could not be saved in this browser.',
      })
    }
```

```
  }

  function addItem(bookId) {
    const existingItem = items.find((item) => item.bookId === bookId)
    const nextItems = existingItem
      ? items.map((item) =>
          item.bookId === bookId
            ? { ...item, quantity: item.quantity + 1 }
            : item,
        )
      : [...items, { bookId, quantity: 1 }]

    saveItems(nextItems)
  }

  function setItemQuantity(bookId, quantity) {
    if (!Number.isInteger(quantity) || quantity < 0) {
      throw new RangeError('Cart quantity must be a non-negative whole number.')
    }

    const nextItems =
      quantity === 0
        ? items.filter((item) => item.bookId !== bookId)
        : items.map((item) =>
            item.bookId === bookId ? { ...item, quantity } : item,
          )

    saveItems(nextItems)
  }

  function removeItem(bookId) {
    saveItems(items.filter((item) => item.bookId !== bookId))
  }

  const clearCart = useCallback(() => {
    saveItems([])
  }, [])

  const itemCount = items.reduce((total, item) => total + item.quantity, 0)

  return (
    <CartContext.Provider
      value={{
        items,
        itemCount,
        storageError,
        addItem,
        setItemQuantity,
        removeItem,
        clearCart,
      }}
    >
      {children}
    </CartContext.Provider>
  )
}
```

Cart context helper

useCart() gives components access to CartProvider and throws a clear error if wired outside the provider.

Complete source: frontend/src/context/cart-context.js | 13 lines

```
 1  import { createContext, useContext } from 'react'
 2
 3  export const CartContext = createContext(null)
 4
 5  export function useCart() {
 6    const cart = useContext(CartContext)
 7
 8    if (!cart) {
 9      throw new Error('useCart must be used inside a CartProvider.')
10    }
11
12    return cart
13  }
```

Database schema

The book table supplies trusted catalogue prices. orders stores buyer, state, total cents, Stripe Session ID, and timestamps. orderItems stores purchased quantities and price snapshots.

Complete source: server/src/db/schema.js | 87 lines

```
import {
  pgTable,
  serial,
  integer,
  text,
  numeric,
  boolean,
  timestamp,
} from 'drizzle-orm/pg-core'

// We need the user table so each order can point at its buyer
import { user } from './auth-schema.js'

// Describe the books table and its columns
export const books = pgTable('books', {
  // Keep the IDs from our existing book data
  id: integer('id').primaryKey(),

  title: text('title').notNull(),
  author: text('author').notNull(),
  category: text('category').notNull(),

  // Store prices with two decimal places
  price: numeric('price', {
    precision: 10,
    scale: 2,
    mode: 'number',
  }).notNull(),

  // Store the image path, rather than the image itself
  image: text('image').notNull(),

  staffPick: boolean('staff_pick').default(false).notNull(),
  featured: boolean('featured').default(false).notNull(),
})

// One row per purchase
export const orders = pgTable('orders', {
  id: serial('id').primaryKey(),

  // Which signed-in user placed this order
  userId: text('user_id')
    .notNull()
    .references(() => user.id),

  // Starts as 'pending'. Only the Stripe webhook may change it to 'paid'
  status: text('status').notNull().default('pending'),

  // Total in cents (1499 means $14.99), because Stripe counts in cents
  totalCents: integer('total_cents').notNull(),

  // Filled in after we create the Stripe Checkout Session
  stripeSessionId: text('stripe_session_id').unique(),

  createdAt: timestamp('created_at').defaultNow().notNull(),

  // Set by the webhook at the moment payment is confirmed
  paidAt: timestamp('paid_at'),
```

```
})

// One row per book inside an order
export const orderItems = pgTable('order_items', {
  id: serial('id').primaryKey(),

  // Which order this line belongs to. Deleting an order deletes its items
  orderId: integer('order_id')
    .notNull()
    .references(() => orders.id, { onDelete: 'cascade' }),

  // Which book was bought
  bookId: integer('book_id')
    .notNull()
    .references(() => books.id),

  quantity: integer('quantity').notNull(),

  // The price in cents at the moment of purchase
  unitPriceCents: integer('unit_price_cents').notNull(),
})

// Make the authentication tables available through this schema file
export {
  user,
  session,
  account,
  verification,
} from './auth-schema.js'
```

Order table migration

SQL for creating the two order tables on a new database. Your already-configured database has them; do not run it there again.

Complete source: server/src/db/migrations/0001_create_orders.sql | 17 lines

```
 1  CREATE TABLE IF NOT EXISTS orders (
 2    id serial PRIMARY KEY,
 3    user_id text NOT NULL REFERENCES "user"(id),
 4    status text NOT NULL DEFAULT 'pending',
 5    total_cents integer NOT NULL,
 6    stripe_session_id text UNIQUE,
 7    created_at timestamp NOT NULL DEFAULT now(),
 8    paid_at timestamp
 9  );
10
11  CREATE TABLE IF NOT EXISTS order_items (
12    id serial PRIMARY KEY,
13    order_id integer NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
14    book_id integer NOT NULL REFERENCES books(id),
15    quantity integer NOT NULL,
16    unit_price_cents integer NOT NULL
17  );
```

Express server route mounting

The checkout router is mounted at /api, which gives it /api/checkout/session and /api/stripe/webhook URLs.

Complete source: server/src/index.js | 65 lines

```
// Load the Express package we installed.
import express from 'express'
import { db } from './db/index.js'
import { books } from './db/schema.js'
import { toNodeHandler } from 'better-auth/node'
import { auth } from './auth.js'
import checkoutRoutes from './checkout-routes.js'

// Node.js helper: convert a file URL into a filesystem path
import { fileURLToPath } from 'node:url'

// Create our Express application.
const app = express()

// Express sends authentication requests to Better Auth
app.all('/api/auth/*splat', toNodeHandler(auth))
app.use('/api', checkoutRoutes)

// Use the hosting provider's port, or 3000 on our laptop.
const PORT = Number(process.env.PORT || 3000)

// Respond when someone requests /api/health.
app.get('/api/health', (req, res) => {
  res.json({
    ok: true,
    message: 'Boox Store server is running',
  })
})

// Return all books stored in the database
app.get('/api/books', async (req, res) => {
  try {
    // Read the books, ordered by their IDs
    const allBooks = await db
      .select()
      .from(books)
      .orderBy(books.id)

    // Send the resulting array to the browser as JSON
    res.json(allBooks)
  } catch (error) {
    console.error(
      'Could not fetch books:',
      error.cause?.code ?? error.code ?? 'Unknown'
    )

    // Tell the browser that the request failed
    res.status(500).json({
      error: 'Could not load books. Please try again.',
    })
  }
})

// Locate the finished React website
const frontendPath = fileURLToPath(
  new URL('../../frontend/dist/', import.meta.url)
)

// Express serves the website, images, CSS and browser JavaScript
app.use(express.static(frontendPath))
```

```
61
62  // Start listening for requests.
63  app.listen(PORT, () => {
64    console.log(`Boox Store API running at http://localhost:${PORT}`)
65  })
```

Local API proxy

During local development, this forwards /api calls from Vite to Express on localhost:3000.

Complete source: frontend/vite.config.js | 14 lines

```
 1  import { defineConfig } from 'vite'
 2  import react from '@vitejs/plugin-react'
 3  import tailwindcss from '@tailwindcss/vite'
 4
 5  export default defineConfig({
 6    plugins: [react(), tailwindcss()],
 7
 8    server: {
 9      proxy: {
10        // Forward API requests to our Express server
11        '/api': 'http://localhost:3000',
12      },
13    },
14  })
```

Application wiring

Loads the catalogue and wraps the header, book grid, and cart in CartProvider.

Complete source: frontend/src/App.jsx | 78 lines

```
import { useEffect, useState } from 'react'
import Header from './components/Header.jsx'
import Hero from './components/Hero.jsx'
import BookGrid from './components/BookGrid.jsx'
import CartSection from './components/CartSection.jsx'
import Events from './components/Events.jsx'
import About from './components/About.jsx'
import Footer from './components/Footer.jsx'
import AccountSection from './components/AccountSection.jsx'
import { CartProvider } from './context/CartContext.jsx'

export default function App() {
  const [books, setBooks] = useState([])
  const [booksLoading, setBooksLoading] = useState(true)
  const [booksError, setBooksError] = useState('')

  useEffect(() => {
    const controller = new AbortController()

    async function loadBooks() {
      try {
        const response = await fetch('/api/books', {
          signal: controller.signal,
        })

        if (!response.ok) {
          throw new Error('Could not load books. Please refresh to try again.')
        }

        const data = await response.json()

        if (!controller.signal.aborted) {
          setBooks(data)
        }
      } catch (error) {
        if (!controller.signal.aborted) {
          setBooksError('Could not load books. Please refresh to try again.')
          console.error('Book request failed:', error.message)
        }
      } finally {
        if (!controller.signal.aborted) {
          setBooksLoading(false)
        }
      }
    }

    loadBooks()

    return () => controller.abort()
  }, [])

  return (
    <CartProvider>
      <div className="min-h-screen bg-[#F2E6D2] text-[#30241C]">
        <Header />

        <main>
          <Hero />
          <AccountSection />
          <BookGrid
```

```
            books={books}
            loading={booksLoading}
            error={booksError}
          />
          <CartSection
            books={books}
            booksLoading={booksLoading}
            booksError={booksError}
          />
          <Events />
          <About />
        </main>

        <Footer />
      </div>
    </CartProvider>
  )
}
```

Safe environment variable names

Only blank values are shown. Your actual server/.env is intentionally not read or included.

Complete source: server/.env.example | 8 lines

```
1
2
3  DATABASE_URL=
4  BETTER_AUTH_SECRET=
5  BETTER_AUTH_URL=http://localhost:3000
6  CLIENT_URL=http://localhost:5173
7  STRIPE_SECRET_KEY=
8  STRIPE_WEBHOOK_SECRET=
```

Server scripts and dependencies

Shows Stripe SDK and the npm test/dev/start scripts used by checkout.

Complete source: server/package.json | 27 lines

```
{
  "name": "server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "node --watch src/index.js",
    "start": "node src/index.js",
    "test": "node --test"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module",
  "dependencies": {
    "@better-auth/drizzle-adapter": "^1.7.12",
    "better-auth": "^1.7.12",
    "dotenv": "^18.0.8",
    "drizzle-orm": "^0.45.4",
    "express": "^5.3.0",
    "postgres": "^3.4.9",
    "stripe": "^23.0.1"
  },
  "devDependencies": {
    "drizzle-kit": "^0.31.11"
  }
}
```

End of checkout code guide

This guide describes the repository checkout implementation and local Stripe test workflow. Test mode is not production readiness: before live payments, decide shipping, tax, fulfilment, refunds, and production webhook/secret configuration.